

DEPARTMENT OF COMPUTER APPLICATIONS
MASTER OF COMPUTER APPLICATIONS (MCA)

Final Year Project - Semester III (2025 - 2026)

ZeroTrustCloud: Implementing and Automating Identity-Centric Zero-Trust Architecture for Cloud-Native Environments

WORK PLAN & PROJECT PROGRESS REPORT

Domain: Cybersecurity & Supply Chain Defense

Proposed Platform: TrustMesh

Current Project Status: **Research & Planning Phase**

PROJECT TEAM MEMBERS

- **Hithaishi S P** (GitHub: HithaishiSP2004)
- **Baire Gowda** (GitHub: bairegowda1003)
 - **Sanjay** (GitHub: sanju722002)

1. Project Overview

Cloud-native applications increasingly rely on distributed workloads, microservices, containers, Kubernetes, and hybrid multi-cloud infrastructure. In such distributed environments, traditional perimeter-based security architectures ('castle-and-moat') are fundamentally obsolete because workloads, microservices, APIs, and operators operate dynamically across disparate networks, clouds, and compute fabrics without static physical perimeters.

ZeroTrustCloud is proposed as a comprehensive cybersecurity research and engineering project focused on designing, implementing, and automating an identity-centric Zero-Trust Architecture (ZTA) tailored specifically for containerized and cloud-native environments.

The central engineering artifact of this project is TrustMesh—a proposed unified Zero-Trust security control plane that integrates workload cryptographic identity, automated Policy-as-Code enforcement, service mesh mutual authentication (mTLS), runtime continuous verification, and end-to-end CI/CD software supply chain integrity into a single cohesive control plane.

The research further investigates the cross-cloud interoperability and practical enforcement of Zero-Trust principles across heterogeneous managed Kubernetes platforms, specifically Amazon Elastic Kubernetes Service (AWS EKS) and Azure Kubernetes Service (Azure AKS).

2. Problem Statement

Modern cloud-native architectures introduce critical security attack surfaces that perimeter defenses fail to mitigate:

- **Dissolution of Network Perimeters:** Dynamic pod scheduling and multi-tenant clusters invalidate IP/port-based security boundaries; network location no longer implies trust.
- **Identity Sprawl & Complexity:** Explosive growth of human identities, ephemeral service accounts, and API access tokens creates unmanageable privilege trees.
- **Over-Privileged Workloads:** Permissive default configurations, excessive IAM bindings, and hardcoded static secrets facilitate lateral privilege escalation upon container escape.
- **Manual Policy Bottlenecks:** Static, human-managed firewall and access control lists do not scale with microservices, resulting in configuration drift and human error.
- **Unprotected East-West Traffic:** Internal cluster communication often flows in plaintext HTTP, exposing internal service calls to interception and spoofing.
- **Supply-Chain Infiltration:** Compromised open-source dependencies, untrusted base images, and unhardened CI/CD runners inject backdoors directly into production runtimes.
- **Configuration Drift & Latent Violations:** Post-deployment configuration tampering easily bypasses one-time static security gates.
- **Fragmented Observability:** Absence of centralized, auditable telemetry correlating authentication events, authorization decisions, and runtime policy breaches.

Consequently, this project investigates how an automated, identity-centric Zero-Trust framework can solve these compounding vulnerabilities across cloud-native environments while maintaining acceptable latency and operational throughput.

3. Project Objectives

To systematically address the stated challenges, the project defines eight core research and development objectives:

- 1. Theoretical & Practical Grounding:** Systematically analyze Zero-Trust Architecture principles (NIST SP 800-207) and map their application to cloud-native microservice ecosystems.
- 2. Identity-Centric Security Modeling:** Architect a cryptographic, least-privilege identity framework unifying human users, service accounts, and ephemeral container workloads.
- 3. Cross-Cloud Workload Identity Federation:** Implement and evaluate federated workload identity bridging Kubernetes Service Accounts with cloud-provider IAM (AWS IAM Roles for Service Accounts - IRSA, and Azure Workload Identity).
- 4. Automated Policy-as-Code Enforcement:** Design and deploy automated declarative security policies using Open Policy Agent (OPA) / Gatekeeper to eliminate manual configuration drift.
- 5. Zero-Trust Service-to-Service Communication:** Establish mutual TLS (mTLS) with cryptographic attestation and Layer-7 authorization policies using Istio service mesh.
- 6. End-to-End Software Supply Chain Defense:** Integrate automated vulnerability scanning, Software Bill of Materials (SBOM) generation, and Cosign cryptographic artifact signing into CI/CD pipelines.
- 7. Centralized Security Telemetry & Visibility:** Construct an integrated observability control plane aggregating authentication logs, policy evaluations, violations, and security audit trails.
- 8. Empirical Performance & Efficacy Benchmarking:** Measure and evaluate latency overhead, throughput impact, and defensive resilience by executing comparative red-team attack scenarios.

4. Work Completed So Far

During the initial phase of the final-year project, the following foundational milestones have been accomplished:

4.1 Project Topic Finalization

The project title was formally finalized as: ZeroTrustCloud: Implementing and Automating Identity-Centric Zero-Trust Architecture for Cloud-Native Environments, situated under the specialized domain of Cybersecurity & Supply Chain Defense.

4.2 Problem Understanding & Domain Scoping

In-depth domain analysis was performed on cloud-native security vectors, focusing on identity sprawl, service-to-service attack surfaces, over-permissioned Kubernetes service accounts, CI/CD pipeline vulnerabilities, and runtime continuous verification.

4.3 Literature Study & Research Review

A comprehensive literature survey was conducted across 10 high-impact research papers covering Zero-Trust Architecture, Kubernetes security, workload identity federation, Policy-as-Code, service mesh mTLS benchmarking, and supply-chain integrity. Each team member analyzed papers to extract methodologies, empirical findings, and identified limitations.

4.4 Initial System Concept: TrustMesh

The initial conceptual blueprint for TrustMesh was formulated as a centralized Zero-Trust control plane comprising six dedicated engines: Identity Engine, Policy Engine, Service Trust Engine, Continuous Verification Engine, Pipeline Security Engine, and Observability & Compliance Engine.

4.5 Technology Exploration

Preliminary feasibility exploration was carried out across candidate technologies including Kubernetes, AWS EKS, Azure AKS, OIDC, SPIFFE/SPIRE, Open Policy Agent (OPA), Istio Service Mesh, mTLS cryptographic handshakes, GitHub Actions, Cosign, Prometheus, and Grafana.

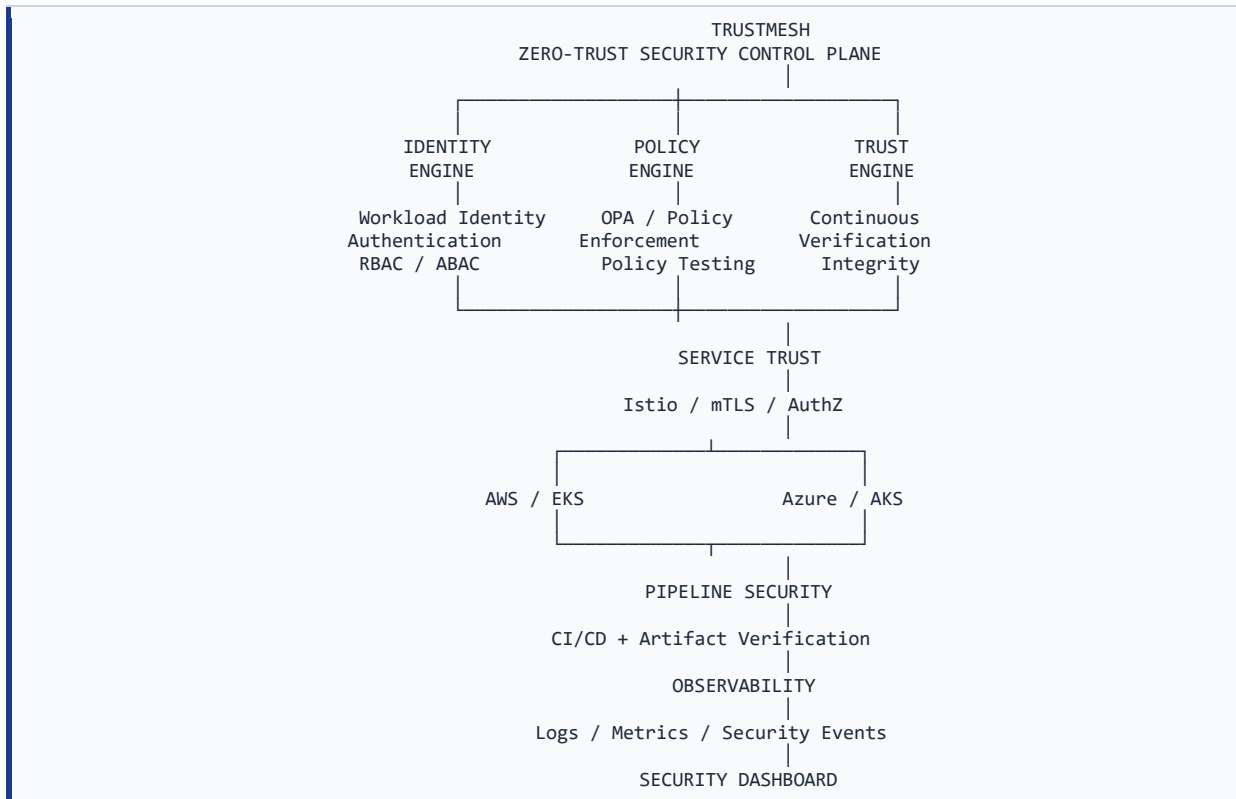
5. Current Work

The project is currently actively progressing within the Research and Planning Phase, executing the following parallel workstreams:

- **Literature Analysis & Gap Extraction:** Synthesizing findings from all reviewed research papers to categorize existing defense mechanisms, experimental benchmarks, and documented architectural shortcomings.
- **Architecture Design Refinement:** Translating research takeaways into an actionable multi-layer technical blueprint for the TrustMesh control plane.
- **Threat Modeling:** Conducting structured threat modeling across cloud-native layers to map potential attack vectors: credential theft, privilege escalation, unauthorized east-west lateral movement, policy tampering, compromised container images, and software supply-chain poisoning.
- **Technology Evaluation & Trade-off Analysis:** Assessing candidate open-source tools against evaluation criteria: security capabilities, multi-cloud compatibility, deployment complexity, latency overhead, and resource consumption.
- **Evaluation & Benchmarking Methodology:** Defining the testbed setup, benchmark metrics, synthetic workload scenarios, and evaluation criteria to quantitatively assess Zero-Trust overhead.

6. Proposed System Blueprint

The proposed TrustMesh architecture is structured as a multi-tier, identity-centric Zero-Trust security control plane spanning workload identity, policy enforcement, encrypted mesh communication, supply-chain validation, and centralized visibility:



Note: This diagram illustrates the proposed high-level system blueprint. Individual component bindings and runtime protocols will be finalized following the comprehensive feasibility study.

7. Proposed Modules

7.1 Identity & Workload Security Engine

Investigates cryptographic, identity-first access control across users, Kubernetes service accounts, and microservice workloads. Explores OIDC federation, SPIFFE/SPIRE workload attestation, dynamic RBAC/ABAC mapping, and the elimination of static long-lived API tokens in favor of short-lived cryptographic x509 SVIDs.

7.2 Policy-as-Code Engine

Provides automated, declarative security policy governance across clusters. Integrates Open Policy Agent (OPA) and Rego policies via Kubernetes admission controllers to validate incoming resource manifests, prevent privilege escalation, enforce resource constraints, and detect runtime policy drift through GitOps versioning.

7.3 Service Trust Engine

Guarantees zero implicit trust across inter-service network traffic. Utilizes Istio Service Mesh to enforce strict mutual TLS (mTLS) with automated certificate rotation, service-level identity verification, and granular layer-7 authorization policies, preventing unauthorized lateral movement.

7.4 Continuous Verification Engine

Maintains persistent runtime surveillance over workload integrity. Performs continuous container image hash attestation, detects unauthorized configuration changes, tracks unexpected system-call deviations, and dynamically recalculates trust scores based on real-time behavior.

7.5 Pipeline & Supply-Chain Security Engine

Secures the software delivery lifecycle from source code commit to container runtime. Implements secretless GitHub Actions runners using OIDC federation, automated vulnerability triage via Trivy, Software Bill of Materials (SBOM) generation via Syft, and image signing and verification using Cosign/Sigstore.

7.6 Observability & Compliance Engine

Aggregates security telemetry across all distributed control points. Collects authentication events, authorization denials, admission rejections, and runtime alerts into a unified dashboard, enabling automated compliance scoring and auditable forensic analysis.

8. Proposed Technology Stack

The candidate technologies selected for evaluating and implementing TrustMesh are summarized below:

Security Architecture Area	Proposed Technologies & Tools
Cloud Infrastructure	Amazon Web Services (AWS), Microsoft Azure
Container Orchestration	Kubernetes (v1.28+)
AWS Managed Kubernetes	Amazon Elastic Kubernetes Service (Amazon EKS)
Azure Managed Kubernetes	Azure Kubernetes Service (Azure AKS)
Control Plane Backend	Python 3.11 / FastAPI (Asynchronous REST API)
Management Dashboard	React 18 / TypeScript / TailwindCSS
Configuration & Audit Store	PostgreSQL (Relational Store with TimescaleDB)
Workload Identity & Auth	OpenID Connect (OIDC) / SPIFFE / SPIRE
Policy-as-Code Engine	Open Policy Agent (OPA) / Rego / Gatekeeper
Service Mesh & Traffic	Istio Service Mesh (Sidecar & Ambient Modes)
Service Communication	Strict Mutual TLS (mTLS) with automated rotation
Supply Chain & Signing	Cosign / Sigstore / Syft SBOM / Trivy Scanner
Continuous Integration	GitHub Actions (Secretless OIDC Federation)
Metrics & Telemetry	Prometheus / Grafana (Zero-Trust Observability)
Log Aggregation	Grafana Loki / OpenTelemetry Collector

Note: The technology stack is actively under evaluation and may be adapted based on empirical prototype benchmarks, API maturity, and cloud quota availability.

9. Research and Evaluation Plan

A distinguishing aspect of this project is its empirical validation strategy. In addition to system implementation, the security efficacy and system overhead will be quantitatively evaluated:

9.1 Security Evaluation Scenarios

Controlled attack scenarios will be executed against test workloads to evaluate detection and blocking capabilities:

- **Unauthorized Service-to-Service Access:** Attempting direct HTTP invocation between microservices without valid mTLS x509 certificates.
- **Privilege Escalation Exploits:** Deploying compromised pods that request root privileges or host-level filesystem access.
- **Invalid Workload Identities:** Presenting forged, expired, or non-federated SPIFFE IDs to service proxies.
- **Policy-as-Code Violation Tests:** Attempting to deploy containers lacking mandatory labels, resource quotas, or security contexts.
- **Untrusted Artifact Ingestion:** Attempting to deploy container images not cryptographically signed by Cosign/Sigstore.

- **Dependency Vulnerability Infiltration:** Injecting known vulnerable open-source packages into the CI pipeline to test scanner blocks.
- **Configuration Drift Detection:** Modifying live Kubernetes runtime configurations out-of-band to verify automated alert triggers.

9.2 Performance Evaluation Metrics

The performance cost of Zero-Trust security layers will be measured under controlled synthetic traffic loads:

- **Request Latency Overhead:** Comparing p50, p95, and p99 millisecond latency profiles with and without Envoy mTLS proxies.
- **Throughput Degradation:** Measuring maximum requests per second (RPS) achievable before saturation.
- **Resource Footprint:** Quantifying CPU and memory consumption added by sidecars, OPA agents, and SPIRE daemons.
- **Admission Controller Latency:** Timing OPA Rego policy evaluation overhead on Kubernetes API manifest admissions.

9.3 Comparative Security Effectiveness Study

The evaluation will contrast a baseline cloud-native setup (Without Zero-Trust Controls) against the hardened TrustMesh architecture (With Zero-Trust Controls) to demonstrate concrete security enhancements versus operational costs.

10. Research Gap Being Investigated

Existing academic literature and commercial tooling predominantly address Zero-Trust in isolation: identity federation is studied independently from CI/CD supply chains, service meshes are analyzed apart from policy governance, and admission controllers operate disconnected from runtime telemetry.

However, real-world cloud security breakdowns occur precisely at the seams between these decoupled systems. The core research gap addressed by ZeroTrustCloud is:

How can workload identity, declarative policy, service mesh mutual trust, supply-chain verification, and continuous monitoring be systematically unified into a cohesive, automated, and performant security control plane for heterogeneous cloud-native environments?

Identity (SPIFFE/OIDC) --> Policy (OPA) --> Workload Trust --> Service Security (mTLS) --> Supply Chain (Cosign) --> Continuous Monitoring

By establishing an integrated identity lifecycle where the artifact signed in CI/CD carries provenance directly into runtime Kubernetes admission and service mesh authorization, TrustMesh eliminates the configuration gaps typical of fragmented point solutions.

11. Next Plan and Remaining Tasks

The project execution roadmap is structured into eight distinct development and research phases:

Phase 1 - Research & Requirements [In Progress]

- Complete systematic analysis across 10 core research papers.
- Formulate detailed functional and non-functional security requirements.
- Construct STRIDE-based cloud-native threat model and benchmark metrics.

Phase 2 - Architecture & Design [Planned]

- Finalize TrustMesh control plane component interactions and data flows.
- Design cryptographic identity federation flows across EKS and AKS.
- Establish database schema for audit logging, security events, and policy drift.

Phase 3 - Cloud & Kubernetes Foundation [Planned]

- Provision development Kubernetes clusters across AWS EKS and Azure AKS.
- Deploy sample microservice architecture representing enterprise distributed workloads.
- Establish cloud IAM and Kubernetes Workload Identity federation.

Phase 4 - Zero-Trust Security Controls Implementation [Planned]

- Deploy SPIRE server and agents for workload cryptographic attestation.
- Configure OPA Gatekeeper with baseline Rego security admission policies.
- Implement Istio Service Mesh with STRICT mutual TLS and Layer-7 authorization policies.

Phase 5 - Supply-Chain Security Pipeline [Planned]

- Configure GitHub Actions with OIDC-based short-lived cloud authentication.
- Implement automated dependency vulnerability scanning and SBOM generation.
- Automate container image signing via Cosign and enforce signature verification at cluster admission.

Phase 6 - Monitoring & Security Dashboard [Planned]

- Ingest security telemetry into Prometheus and central logging infrastructure.
- Develop web dashboard displaying real-time security posture, violations, and identity state.
- Implement automated alerting on policy tampering and anomalous service behaviors.

Phase 7 - Testing & Empirical Evaluation [Planned]

- Execute simulated attack scenarios to assess defensive efficacy.
- Conduct load testing to measure latency, throughput, and resource overhead.
- Analyze comparative data: baseline architecture versus ZeroTrustCloud.

Phase 8 - Finalization & Documentation [Planned]

- Synthesize research data, empirical charts, and security analysis.
- Complete final project dissertation and technical project documentation.
- Author research publication manuscript and prepare final demonstration and viva defense.

12. Expected Final Outcome

Upon completion, the project will deliver a fully documented, empirical research study and a functional open-source prototype demonstrating:

- **Cryptographic Workload Identity:** Automated issuance of short-lived, rotatable cryptographic identities without hardcoded secrets.
- **Automated Policy-as-Code:** Continuous policy enforcement at cluster admission and runtime without manual review bottlenecks.
- **Zero Implicit Network Trust:** 100% encrypted, authenticated, and authorized east-west microservice traffic using strict mTLS.
- **Verifiable Software Supply Chain:** Guaranteed deployment of only verified, SBOM-documented, and cryptographically signed artifacts.
- **Multi-Cloud Security Uniformity:** Consistent security policies and workload identity verification across both AWS EKS and Azure AKS.
- **Centralized Posture Dashboard:** Real-time visibility into cluster security health, policy violations, and auditable security decisions.
- **Empirical Overhead Benchmarks:** A quantitative evaluation detailing latency, throughput, and compute resource trade-offs.
- **Complete Academic Dissertation:** Thoroughly documented research paper and dissertation meeting all MCA final-year curriculum standards.

13. Expected Project Completion Plan

The 10-stage execution milestones and projected timelines are tabulated below:

Stage	Major Activities & Milestones	Expected Status
Stage 1	Literature Review, Research Paper Analysis & Gap Identification	Current
Stage 2	Requirements Formulation, Threat Modeling & Evaluation Metrics	Next
Stage 3	System Architecture Blueprinting & Technology Stack Finalization	Next
Stage 4	Multi-Cloud Infrastructure & Kubernetes Cluster Foundation	Planned
Stage 5	Identity Federation & Policy-as-Code Engine Implementation	Planned
Stage 6	Service Mesh (mTLS) & Supply-Chain Security Integration	Planned
Stage 7	Centralized Observability, Auditing & Posture Dashboard	Planned
Stage 8	Controlled Penetration Testing & Empirical Performance Evaluation	Planned
Stage 9	Data Analysis, Architecture Refinement & Report Writing	Planned
Stage 10	Final Prototype Demonstration, Project Viva & Dissertation Submission	Planned

14. Current Project Status

Overall Project Status: Research & Planning Phase - In Progress

Completed Milestones

- **[Completed]** Project topic finalized and approved for MCA final-year implementation.
- **[Completed]** Comprehensive problem statement and research scope defined.
- **[Completed]** Eight core project research and engineering objectives identified.
- **[Completed]** Literature survey of 10 relevant peer-reviewed research papers collected and reviewed.
- **[Completed]** Initial conceptual architecture of TrustMesh control plane defined.
- **[Completed]** Six foundational system modules identified and scoped.
- **[Completed]** Preliminary feasibility evaluation of core cloud-native technologies completed.

Currently In Progress

- --> Literature synthesis and extraction of documented technical research gaps.
- --> STRIDE-based cloud-native threat modeling across workload and pipeline layers.
- --> Formal functional and non-functional requirements specification.
- --> Refinement of the multi-cloud TrustMesh architecture and interface specifications.
- --> Empirical benchmarking methodology and test scenario design.
- --> Candidate toolchain selection and cluster resource sizing.

Immediate Next Steps

- --> Finalize technical architecture and communication protocols.
- --> Establish development Kubernetes environment across AWS and Azure.
- --> Begin incremental implementation of Phase 3 & Phase 4 modules.
- --> Deploy baseline workloads and conduct initial latency and overhead baselining.

Conclusion

The ZeroTrustCloud project is progressing systematically from the preliminary research and planning phase toward concrete architectural formalization and prototype implementation.

The comprehensive literature study across 10 foundational research papers has established the technical basis for the project, identifying critical security gaps in current cloud-native architectures where identity, policy, service encryption, and supply-chain controls remain dangerously fragmented.

By implementing and validating the unified TrustMesh control plane, the project aims to demonstrate that automated, identity-centric Zero-Trust principles can be practically deployed across heterogeneous Kubernetes environments without imposing unacceptable operational complexity or performance degradation.

Development will proceed iteratively according to the phased completion plan, ensuring that each security module is verified independently before being integrated into the end-to-end framework.